Gestión de proyectos de red con enfoque ágil y NetDevOps

Documentación del curso · Sesiones 1 a 3

Iván Osuna Ayuste

Noviembre de 2026

Índice

1 Presentación del documento

2 Evolución de la gestión de proyectos IT

2.1 De la anarquía a la ingeniería del software

2.2 El péndulo hacia la burocracia

2.3 La metodología en cascada

2.4 El control del proyecto en el modelo tradicional

3 Metodologías ágiles

3.1 La idea central: entrega incremental

3.2 Hitos frente a iteraciones

3.3 El sistema funcionando como medida del progreso

3.4 El coste oculto de la agilidad

3.5 El agujero negro de la visibilidad

4 Las pruebas

4.1 Para qué sirven

4.2 Niveles de prueba

4.3 Error, defecto y fallo

4.4 Anatomía de una prueba

4.5 Los principios FIRST

4.6 Pruebas que diagnostican

5 DevOps y NetDevOps

5.1 Una cultura, no una metodología

5.2 El ciclo DevOps y su grado de automatización

5.3 Qué es automatizar

5.4 Por qué automatizar

5.5 Automatizar es caro y difícil

5.6 Tareas frente a procesos

5.7 La foto final: integración y despliegue continuos en red

6 Infraestructura como código

6.1 Definición

6.2 Paradigmas: imperativo frente a declarativo

6.3 Idempotencia

6.4 El programa como catálogo de recursos

6.5 Lenguajes de propósito general y de dominio específico

6.6 Versionado semántico

7 Control de versiones con Git

7.1 El commit es una fotografía

7.2 Las ramas

8 Coste, mantenibilidad e inteligencia artificial
- 8.1 El coste que importa: TCO
- 8.2 Mantenible y evolucionable
- 8.3 La IA agéntica

9 Implantación en la organización
- 9.1 Un cambio lento y cultural
- 9.2 Evolución de las estructuras organizativas
- 9.3 ITIL

10 Caso práctico: migración de ACL
- 10.1 Planteamiento
- 10.2 Opción 1: enfoque tradicional
- 10.3 Opción 2: enfoque ágil
- 10.4 Las iteraciones siguientes: automatizar
- 10.5 El papel de la gestión

11 Gestión de riesgos
- 11.1 Cálculo del riesgo
- 11.2 Ejemplo: la capacidad de los cortafuegos

12 El modelo de trabajo del NOC
- 12.1 Los cinco planos
- 12.2 Tipos de trabajo
- 12.3 Estados y bloqueo
- 12.4 Equivalencias con Redmine
- 12.5 Equivalencias con ITIL

13 Cuadros de mando
- 13.1 Datos no es entender
- 13.2 Cuadro operativo y BI
- 13.3 Tres niveles de información
- 13.4 Métricas automáticas y de disciplina
- 13.5 El cuadro de mando del proyecto de migración
- 13.6 Un cuadro para cada audiencia
- 13.7 Reglas de diseño
- 13.8 Aterrizaje en Redmine, por fases

14 Ideas clave

15 Glosario

1 Presentación del documento

Este documento recoge, de forma ordenada y formal, los contenidos tratados en las tres primeras sesiones del curso de gestión de proyectos para el equipo de operación de red (NOC). Parte de las notas tomadas en clase y de dos documentos de referencia que acompañan al curso:

- **Cuadros de mando en un servicio NOC** (`dia3/cuadros de mando.md`).
- **Modelo de trabajos vivos en un NOC**, con las equivalencias con Redmine e ITIL (`dia3/tipos de trabajos y estados.md`).

El hilo conductor es una pregunta muy concreta que cualquier responsable de proyecto se hace a diario: **¿cómo voy?** Todo lo que sigue —metodologías ágiles, pruebas, automatización, infraestructura como código, control de versiones, modelo de trabajo y cuadros de mando— son piezas para poder contestarla con datos y no con impresiones.

Allí donde el documento va más allá de lo explicado en clase, para completar un concepto o darle una base más formal, se indica con un recuadro **Ampliación**.

> **Cómo leer este documento.** Los capítulos 1 a 4 explican *por qué* ha cambiado la forma de gestionar proyectos. Los capítulos 5 a 7 explican *con qué* herramientas y conceptos técnicos se sostiene ese cambio. Los capítulos 8 a 12 aterrizan todo en el proyecto real de migración de ACL y en cómo gobernarlo: implantación, riesgos, modelo de trabajo y cuadro de mando.

2 Evolución de la gestión de proyectos IT

2.1 De la anarquía a la ingeniería del software

En los primeros años de la informática (décadas de 1950 y 1960) no existía una forma establecida de gestionar los proyectos. Cada persona hacía lo que podía. Faltaban metodologías de gestión, estándares de calidad, procesos definidos, herramientas, arquitecturas de referencia, patrones y principios de diseño.

El resultado fue lo que se conoce como la **crisis del software**, a finales de los años 60: plazos que se incumplían por mucho, sistemas que había que rediseñar por completo, errores frecuentes y costes desorbitados.

La respuesta fue una disciplina nueva, la **ingeniería del software**, cuyo propósito era aportar todo lo que faltaba: poner orden en los proyectos.

2.2 El péndulo hacia la burocracia

La reacción fue excesiva. Se pasó de la anarquía al extremo contrario:

- burocracia y documentación exhaustiva;
- procesos rígidos y pesados;
- departamentos centralizados y jerárquicos;
- decisiones hipercentralizadas;
- poca flexibilidad y mala adaptación al cambio.

Muchas organizaciones siguen funcionando hoy en este modelo.

2.3 La metodología en cascada

La metodología más extendida en esta etapa fue la **cascada** (*waterfall*) y sus variantes (modelo en V, espiral). El proyecto avanza por fases secuenciales:

1. Toma de requisitos.
2. Análisis y diseño de la solución.
3. Implementación.
4. Pruebas.
5. Despliegue.
6. Mantenimiento.

Su problema de fondo es que **se adapta muy mal al cambio de requisitos**, y en el mundo de las TIC los requisitos cambian, y mucho, por tres motivos:

- el cliente no lo tenía claro desde el principio;
- el negocio o el mercado cambian mientras dura el proyecto;

- quien toma los requisitos se equivoca, porque es humano.

2.4 El control del proyecto en el modelo tradicional

En el modelo tradicional se hace una **planificación exhaustiva al inicio**, normalmente con herramientas como MS Project y diagramas de Gantt, hasta el nivel de tarea, persona y hora. Las tareas se agrupan en **hitos** con fecha.

Cuando llega la fecha de un hito y una de sus tareas no está terminada, se dispara la alarma y se **replanifica el hito**: se mueve la fecha. El hito es un elemento de **control interno**, y lo que importa es el paquete de tareas, no la fecha.

El efecto acumulado es conocido: una desviación arrastra a las fases siguientes, y durante buena parte del proyecto el cliente no recibe nada.

3 Metodologías ágiles

3.1 La idea central: entrega incremental

Las metodologías ágiles entran en el mundo IT a través del desarrollo de software, aunque su origen está en la industria manufacturera (el sistema de producción de Toyota). Hoy se aplican a todo tipo de proyectos. Su idea principal es:

> ***El proyecto se entrega al cliente de forma incremental.***

En lugar de esperar a tenerlo todo, cada poco tiempo se entrega lo que se tiene hasta ese momento. Esto lo cambia todo.

Para el cliente	Para quien ejecuta el proyecto
Empieza a obtener valor desde el principio.	Recibe *feedback* temprano y barato: un cambio afecta solo a lo entregado, no al sistema completo.
Se familiariza con el sistema desde el primer momento.	Puede replanificar prioridades y recursos según ese *feedback*.
Puede corregir el rumbo pronto.	Trabaja con objetivos más pequeños y manejables.
	Tiene un control mucho mejor del proyecto y de sus tiempos.

El principal inconveniente percibido —las molestias al cliente por recibir entregas frecuentes— es gestionable.

3.2 Hitos frente a iteraciones

En un enfoque ágil **no se hace una planificación exhaustiva al inicio**. No se intenta adivinar el futuro; se planifica, como mucho, la siguiente entrega o las dos siguientes. La unidad de planificación es la **iteración** (en Scrum, *sprint*).

	Hito (tradicional)	Iteración (ágil)
Horizonte	Todo el proyecto, planificado al inicio.	Una o dos iteraciones a la vista.
Naturaleza	Control interno.	Control externo: es una **entrega al cliente**.
Lo que manda	El paquete de tareas.	**La fecha.**
Si una tarea no está terminada	Se mueve la fecha del hito.	Se entrega lo que hay; la tarea pasa a la siguiente iteración.
El retraso	Queda oculto tras la replanificación.	Queda **evidenciado** y no bloquea el trabajo nuevo.

3.3 El sistema funcionando como medida del progreso

Uno de los principios del Manifiesto Ágil dice:

> *El software funcionando es la medida principal de progreso.*

Llevado a nuestro terreno: **el sistema funcionando es la medida principal de progreso**. La frase no habla de cómo construir, sino de **cómo medir** el avance de un proyecto: es, literalmente, un indicador de cuadro de mando.

Si se incluyó en el manifiesto es porque antes no se hacía así. Antes, el avance se medía contando cuántas tareas estaban *supuestamente* completadas, según lo que contaba el propio equipo.

¿Y quién dice que el sistema funciona adecuadamente? **No el cliente.** Nadie compra un coche a un vendedor que le dice «llévatelo, pruébalo y ya me dices». Al cliente le tiene que llegar un sistema que funciona; eso es responsabilidad íntegra de quien lo construye.

La analogía del sastre lo ilustra: cuando el sastre entrega el traje, el traje está terminado —botones cosidos, costuras que no se sueltan—. Otra cuestión es si al cliente le sienta bien: que se tomaran mal las medidas (requisitos) o que el cliente haya engordado (cambio de requisitos). El cliente acepta o no el traje, pero lo que nos importa en la gestión es el año que dura el proyecto, cuando la pregunta es *¿cómo voy?*

> ***Quien dice que el sistema funciona son las pruebas.***

3.4 El coste oculto de la agilidad

Si cada iteración es una entrega a producción, cada iteración exige:

- **pruebas de verdad**, no las que uno hace «por encima» en su entorno;
- **repetir las pruebas de todo lo anterior** (pruebas de regresión), porque al tocar cosas se puede haber roto algo;
- **un despliegue en explotación**, con su esfuerzo y su riesgo.

En la iteración 1 se prueban las tareas 1 y 2; en la iteración 2, las tareas 3, 4 y 5 *y de nuevo* la 1 y la 2; en la iteración 3, la 6 y la 7 y todas las anteriores. Con el modelo tradicional se probaba y se desplegaba una vez, al final. Ahora las pruebas y los despliegues se multiplican.

¿De dónde sale el tiempo, el dinero y la gente para tanto? Solo hay una respuesta: **automatizar**. Sin automatización no hay forma de aplicar metodologías ágiles de manera eficiente.

3.5 El agujero negro de la visibilidad

En un proyecto tradicional, la cantidad de trabajo visible que se entrega es alta al principio (documentos de requisitos, planes) y al final (el sistema entregado), pero entre medias hay un largo periodo en el que no sale nada que el cliente pueda usar ni que permita medir el avance real.

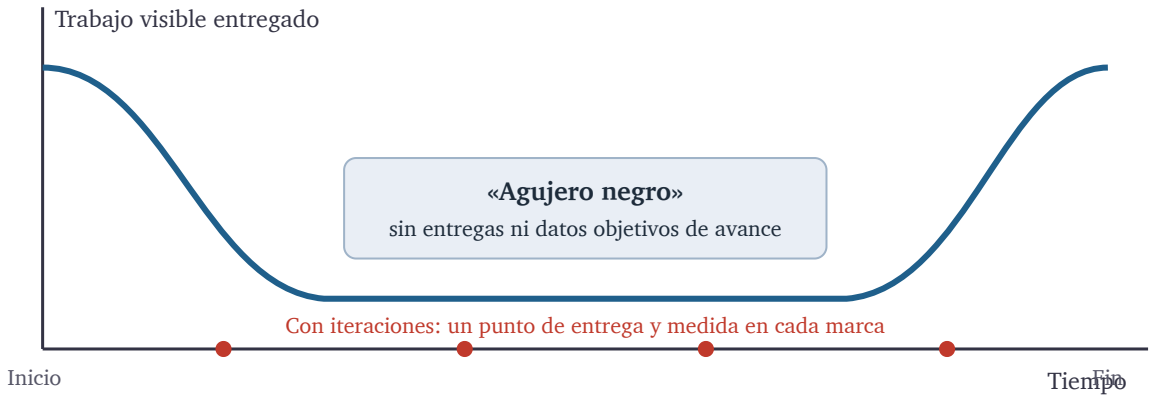


Figura 1. En cascada, el periodo central del proyecto no ofrece información objetiva. Las iteraciones convierten ese periodo en una sucesión de entregas medibles.

4 Las pruebas

4.1 Para qué sirven

Las pruebas cumplen tres funciones:

1. **Asegurar el cumplimiento de los requisitos**, funcionales y no funcionales.
2. **Saber qué tal va el proyecto**: son la fuente objetiva del avance.
3. **Detectar el mayor número posible de defectos antes de que lleguen a producción.**

4.2 Niveles de prueba

Una forma de clasificar las pruebas es por el contexto en que se realizan:

Nivel	Qué comprueba
Unitarias	Un elemento aislado (una regla, una tarea de un *playbook*, una función).
De integración	Que varios elementos funcionan correctamente juntos.
De sistema	Que el sistema completo cumple los requisitos funcionales y no funcionales.
De aceptación	Que el sistema le sirve al cliente: si «le sienta bien el traje».

En el modelo tradicional, las pruebas de aceptación se hacían una vez, con la entrega. En el ágil se hacen en cada entrega.

En el caso del proyecto de migración aparecen, además, las pruebas **no funcionales** de capacidad:

Tipo	Qué mide
Rendimiento	Cuánto impacta cada tipo de tráfico en el equipo y cómo evoluciona con más peticiones.
Carga	El comportamiento con el volumen de tráfico pesado esperado.
Estrés	El comportamiento con tráfico sostenido, o por encima de lo esperado, hasta encontrar el límite.

Y las pruebas de **humo** (*smoke tests*): un conjunto reducido de comprobaciones que se ejecutan justo después de un despliegue para confirmar que lo esencial funciona.

4.3 Error, defecto y fallo

El vocabulario de las pruebas, que se extrapola a cualquier ámbito, distingue tres conceptos:

Término	Qué es
Error	La equivocación que comete una persona.
Defecto	La huella que ese error deja en el producto (una regla mal escrita, un MTU mal configurado).
Fallo	La desviación del comportamiento esperado que se manifiesta al usar el producto.

No se puede arreglar un error ni un fallo: son hechos pasados, y de un fallo solo se pueden gestionar sus consecuencias. **Lo único que se puede corregir es el defecto**, que es lo que permanece en el producto. Encontrar y corregir el defecto es la depuración (*debugging*).

4.4 Anatomía de una prueba

Una fotografía no es una prueba. Una prueba se define con tres elementos:

1. **Contexto**: el entorno y el estado de partida.
2. **Acción**: lo que se ejecuta y se quiere probar.
3. **Resultado esperado**: lo que debe ocurrir. Esto es la comprobación en sí.

Una forma habitual de escribirla es el formato **Dado / Cuando / Entonces** (*Given / When / Then*):

> **Prueba de etiquetado de un equipo**
>
> - **Dado** un equipo A que debe etiquetarse con la información X,
> - **cuando** se etiqueta el equipo y se le hace una fotografía,
> - **entonces** en la fotografía debe aparecer el equipo A con su etiqueta visible y con los datos X correctos.
>
> Si la comprobación la hace una persona mirando la foto, es una **prueba manual**. Si la hace una IA de visión, es una **prueba automatizada**.

> **Prueba de conectividad**
>
> - **Dado** un entorno con un equipo con dirección 192.168.1.100 y otro con dirección 192.168.1.102,
> - **cuando** se ejecuta desde 192.168.1.102 un `ping` a 192.168.1.100 con un único paquete,
> - **entonces** el comando debe terminar con código de salida 0 y debe informar de 1 paquete recibido.

El contexto puede ser el entorno de producción, uno simulado o un clon del de producción.

4.5 Los principios FIRST

Las pruebas bien diseñadas cumplen los principios FIRST:

Principio	Significado
Fast	Rápidas: si tardan, no se ejecutan con la frecuencia necesaria.
Independent	Independientes: el resultado de una no depende de otra.
Repeatable	Repetibles: mismo contexto, mismo resultado.
Self-validating	Autovalidables: dan un resultado binario (pasa / no pasa) sin interpretación humana.
Timely	Oportunas: se definen a tiempo, idealmente antes o a la vez que el trabajo que comprueban.

4.6 Pruebas que diagnostican

Supongamos que se quiere comprobar que una aplicación web (`app1.miempresa.com`, en 192.168.2.200, puerto 443) está accesible. La aplicación expone una ruta `/health` que devuelve un código 200 cuando está desplegada. Una prueba posible es:

```
curl https://app1.miempresa.com/health   →  ¿código 200?
```

Si la prueba pasa, es suficiente. **El problema es cuando falla**: no dice dónde está el problema. Puede ser el servidor caído, el DNS caído o mal configurado, el proceso web parado, el cortafuegos de la máquina, el de la red, la conectividad, el enrutamiento… Para diagnosticar hace falta **una prueba por cada causa posible**.

Esto tiene una consecuencia práctica importante:

- En el **entorno de pruebas**, antes del paso a explotación, se ejecutan todas.
- En la **monitorización de explotación**, cada pocos segundos basta con la prueba general (el `curl`).
- **Cuando algo falla en explotación**, se necesitan todas las pruebas de diagnóstico. Si no están definidas, habrá que escribirlas bajo presión, con el servicio caído y consumiendo el presupuesto de indisponibilidad.

Estos son los costes que no aparecen en un diagrama de Gantt.

> **Ampliación: pruebas negativas.** En reglas de filtrado no basta con comprobar que lo permitido pasa (pruebas positivas). También hay que comprobar que lo que debe estar cerrado sigue cerrado (pruebas negativas o de control). En una migración de ACL, una prueba negativa verifica que la traducción no ha abierto más de lo que había y que la denegación implícita sigue funcionando. Un conjunto de pruebas que solo contiene positivas no detecta el defecto más peligroso de una migración: abrir de más.

5 DevOps y NetDevOps

5.1 Una cultura, no una metodología

DevOps no es una herramienta ni una metodología: es una cultura, una forma de entender cómo evoluciona un producto, que persigue **automatizar todo lo que se pueda**. Es la evolución del ciclo de vida de desarrollo tradicional, con dos cambios de enfoque:

- Da la misma importancia a la **operación** del producto que a su desarrollo.
- Ya no se habla de **fases** secuenciales por las que pasa el producto, sino de **tipos de trabajo** que se encadenan y que ocurren simultáneamente y para siempre. El producto está en todas las etapas a la vez.

En el modelo tradicional, cada versión del producto era un proyecto. En DevOps, el producto evoluciona de forma continua.

NetDevOps es la aplicación de esta cultura a la operación de redes.

5.2 El ciclo DevOps y su grado de automatización

Trabajo	Descripción	¿Automatizable?
Plan	Planificación.	Poco.
Code	Desarrollo: escribir el *playbook*, la configuración.	Cada vez más (con ayuda de IA).
Build	Empaquetado.	Totalmente.
Test	Diseño de las pruebas / ejecución de las pruebas.	Diseño: cada vez más. Ejecución: totalmente.
Release	Liberación: el acto formal de aprobar una versión para su paso a explotación.	Bastante.
Deploy	Despliegue en explotación.	Bastante.
Operate	Operación del sistema.	Bastante.
Monitor	Monitorización.	Totalmente.

Sobre este ciclo se definen tres prácticas encadenadas:

- **Integración continua (CI)**: automatiza desde el desarrollo hasta las pruebas.
- **Entrega continua**: añade la liberación; el resultado queda listo para desplegar.
- **Despliegue continuo (CD)**: añade el despliegue en explotación sin intervención manual.

5.3 Qué es automatizar

Automatizar es construir una máquina —o cambiar el comportamiento de una mediante un programa— para que haga lo que antes hacía una persona con sus manos. Una lavadora automatiza el lavado; sus programas (delicado, algodón a 60 °C) cambian su comportamiento.

En las TIC la máquina ya existe: el ordenador. Por eso, **en nuestro mundo, automatizar es programar**, es decir, desarrollar software.

El trabajo del administrador de red ha cambiado: antes entraba en un equipo y cambiaba su configuración; hoy su trabajo es **crear programas que entren en los equipos y los configuren**.

En una red se trabaja en dos planos:

- **Plano lógico**: todo el trabajo en este plano es automatizable. Lo que queda son personas escribiendo los programas que lo automatizan.
- **Plano físico**: cada vez se virtualiza más, y en cuanto se virtualiza pasa a ser plano lógico. Aun así, siempre habrá cables que tender y equipos físicos que instalar.

5.4 Por qué automatizar

- **Rapidez**: las tareas se hacen en menos tiempo.
- **Consistencia y estandarización**: el mismo trabajo se hace siempre igual.
- **Menos errores**: las personas se equivocan; un programa correcto ejecuta siempre lo mismo.
- **Repetición**: toda tarea debería ejecutarse al menos dos veces —en un entorno previo y en explotación—, así que automatizarla siempre compensa a medio plazo.
- **Control de versiones**, si se hace bien (ver capítulo 6).

5.5 Automatizar es caro y difícil

Conviene no caer en la ingenuidad:

- **Es caro.** Cuesta mucho más esfuerzo crear un programa que haga algo que hacerlo a mano una vez.
- **Es difícil.** Exige un conocimiento muy profundo de lo que se quiere automatizar. Cuando se trabaja a mano, se aprende mientras se hace; cuando se automatiza, **el conocimiento hay que traerlo ya de casa**. Herramientas como Ansible o Terraform no ponen las cosas fáciles: exigen conocer a fondo el sistema que se automatiza.

Además, al automatizar desaparece la persona que revisaba el trabajo. Si nadie mira, **las pruebas tienen que ser buenas y fiables**: son la única garantía que queda.

5.6 Tareas frente a procesos

- **Tarea**: actividad concreta, con inicio y fin claros, que realiza una persona o un sistema.
- **Proceso**: conjunto de tareas relacionadas que persiguen un objetivo común (un producto).

> ***Para automatizar un proceso, antes hay que haber automatizado todas las tareas que lo componen.***

El ejemplo de la persiana lo muestra. Si se cambia la cuerda por un motor y un botón, se ha automatizado una **tarea** (subir y bajar), pero no el **proceso**: sigue haciendo falta una persona que decida cuándo pulsar el botón. Pulsar el botón es *ejecutar el programa*. Si se añade un sensor de luz y una centralita que decide cuándo subir o bajar y «pulsa el botón», entonces sí se ha automatizado el proceso.

La automatización avanza por fases:

1. Se automatizan tareas sueltas, que siguen necesitando a una persona para lanzarlas.
2. Cuando todas las tareas de un proceso están automatizadas, se automatiza el proceso completo, reduciendo o eliminando la intervención humana.

5.7 La foto final: integración y despliegue continuos en red

Fase 1. Integración continua.

1. Una persona escribe un programa, preferiblemente en lenguaje declarativo (por ejemplo, un *playbook* de Ansible), que deja la infraestructura en una versión determinada.
2. Lo sube al sistema de control de versiones, a la rama de desarrollo.
3. Un *pipeline* de integración continua, automáticamente:
   - prepara un entorno de pruebas, a ser posible **efímero** (de usar y tirar);
   - aplica en él el programa descargado del repositorio;
   - ejecuta las pruebas definidas;
   - produce un **informe de pruebas en tiempo real**.

Ese informe es el que dice cómo va el proyecto:

> El proyecto tiene 90 tareas. Hay pruebas definidas para 53. De ellas, 37 pasan y 16 fallan. Respecto a la semana pasada se han definido 7 pruebas nuevas y hay 9 pruebas más que pasan.

Eso es visibilidad real del estado del proyecto, la base para tomar decisiones.

Fase 2. Despliegue continuo. Cuando las pruebas dan verde, el cambio se aplica en explotación de forma automatizada y controlada, y a continuación se ejecutan las pruebas de humo que confirman el despliegue.

Herramientas habituales para automatizar procesos: **Jenkins**, **GitLab CI/CD** y **Ansible AWX / Red Hat Ansible Automation Platform** (antes Ansible Tower).

6 Infraestructura como código

6.1 Definición

Infraestructura como código (IaC) no es escribir un programa que configure un equipo. Es **tratar la infraestructura como si fuera un programa**: definirla y gestionarla con las mismas prácticas con que se gestiona el software. Y lo primero de todo es **someterla a control de versiones**.

¿Para qué sirve versionar la infraestructura?

- Para saber **dónde estoy**.
- Para saber **de dónde vengo** y poder auditar los cambios.
- Para poder **volver atrás** si algo sale mal.
- Para **comparar** versiones y ver qué ha cambiado, no solo en la documentación sino en los recursos efectivos.
- Para saber **quién** introdujo un cambio, **cuándo** y **por qué**.
- Para **deshacer cambios concretos**.

El mundo del software, cuyos proyectos están entre los más complejos que existen, lleva décadas trabajando así. Para la gestión de proyectos de infraestructura esto es clave.

6.2 Paradigmas: imperativo frente a declarativo

Un **paradigma de programación** es una forma de usar un lenguaje para comunicarse —en este caso, con la máquina—. No es exclusivo de la programación: también existe en el lenguaje natural.

Imperativo. Se dan órdenes:

> *Menchu, pon una silla debajo de la ventana.*

Todos los lenguajes tradicionales siguen en mayor o menor medida este paradigma. El problema aparece cuando se quiere que el resultado sea siempre el mismo, sea cual sea la situación de partida. Entonces hay que prever cada caso:

> *Menchu: si debajo de la ventana hay algo que no sea una silla, quítalo. Si no hay silla, y no tienes una, ve a IKEA y compra una. Después, pon la silla debajo de la ventana.*

El programa se complica a gran velocidad.

Declarativo. Se describe cómo deben ser las cosas:

> *Menchu, debajo de la ventana tiene que haber una silla. Es tu responsabilidad.*

El impacto es enorme: **se delega en Menchu la responsabilidad de alcanzar el estado final**. Con el lenguaje imperativo se piden tareas; con el declarativo se describe el estado deseado.

6.3 Idempotencia

La idempotencia es una propiedad matemática: una operación es idempotente si aplicarla varias veces produce el mismo resultado que aplicarla una vez. Por ejemplo, multiplicar por 1 o el valor absoluto: $|-4| = 4$, y $||-4|| = 4$.

En la automatización significa:

> ***Un programa es idempotente si, con independencia del estado inicial del sistema, siempre lo lleva al mismo estado final.***

Conseguirlo con lenguajes imperativos es muy complejo: obliga a llenar el programa de condiciones. **El lenguaje declarativo ya implica idempotencia.** Ansible y Terraform funcionan con este paradigma.

La idempotencia es lo que permite:

- **volver a un estado previo** (*rollback*) a una versión estable y conocida;
- **reparar** cambios o errores introducidos en el sistema;
- **llegar siempre a la versión deseada** del sistema.

6.4 El programa como catálogo de recursos

Un programa declarativo de infraestructura se convierte en un **catálogo de recursos**:

> En mi red quiero:
>
> - un router;
> - un switch conectado al router;
> - el router conectado a una red con salida a internet;
> - el switch operando la red 192.168.0.0/16;
> - una redirección de puertos (NAT) en el router: peticiones externas al 8080 → 192.168.2.17:80.

Ese catálogo, en sus versiones 1, 2, 3…, es la definición de la red lógica. Basta con decir «aplica la v1», «aplica la v2», «aplica la v3» o «vuelve a aplicar la v2». El programa que se ejecuta para aplicar la v2 es el mismo la primera vez que la segunda, aunque el estado de partida sea distinto. Como es idempotente, siempre se llega a la v2, incluso si se parte de la v8. **Esto resuelve el versionado, la vuelta atrás y la coherencia del estado final.**

Además, ese catálogo guarda una relación casi uno a uno con los **requisitos** del sistema.

Para que la herramienta pueda operar un equipo concreto tiene que saber cómo hacerlo. Por eso los fabricantes (Cisco, entre otros) publican **módulos** de Ansible y **providers** de Terraform que «le explican a Menchu» cómo operar sus equipos.

6.5 Lenguajes de propósito general y de dominio específico

	Ejemplos	Analogía
Propósito general	Bash, Python, Java, C.	El español: sirve para todo.
Dominio específico	Ansible, Terraform, SQL, la notación matemática, el silbo gomero.	Un lenguaje hecho para un ámbito concreto.

«2 + 5 = 7» y «dos más cinco son siete» dicen lo mismo; el lenguaje específico es más compacto y preciso en su dominio, y no sirve fuera de él.

6.6 Versionado semántico

En software, y hoy también en infraestructura, se usa el **versionado semántico** (*semver*): `vMAJOR.MINOR.PATCH`, por ejemplo `v1.4.3`.

Componente	Cuándo sube	Ejemplo en red
MAJOR	Cambios que rompen la compatibilidad con versiones anteriores; básicamente, cuando se **quita** algo, con o sin reemplazo.	Cambiar el protocolo de una VPN: quienes la usaban no pueden seguir haciéndolo sin adaptarse.
MINOR	Nueva funcionalidad, o marcar funcionalidad como obsoleta (*deprecated*).	Configurar un NAT para un servicio nuevo.
PATCH	Corrección de defectos.	Subir un MTU demasiado bajo que impedía la comunicación de los sistemas superiores. Es un defecto: la red existe para permitir esa comunicación.

La funcionalidad la marca la pareja `MAJOR.MINOR`. Si la corrección `1.4.1` sale mal, se vuelve a `1.4.0`. Antes de actualizar de `1.4.1` a `1.5.0` se revisa qué cambios incorpora para evaluar el impacto. Desde el punto de vista de gestión, **los cambios MAJOR son los que más problemas dan y donde hay que poner el foco.**

Junto al versionado se mantiene un **registro de cambios** (*changelog*): qué se ha añadido, modificado o corregido en cada versión, para poder evaluar el impacto de actualizar.

7 Control de versiones con Git

7.1 El commit es una fotografía

Git se basa en el concepto de **commit**: una **fotografía completa del proyecto en un momento dado** (una instantánea, una copia de seguridad). No es un paquete de cambios —esa confusión viene de sistemas de control de versiones más antiguos, que sí funcionaban así—. Git permite comparar el estado completo entre dos commits y ver exactamente qué ha cambiado.

7.2 Las ramas

Si Git fuera solo una secuencia de commits, sería un sistema de copias de seguridad. Lo que lo distingue es que maneja **varias secuencias en paralelo: las ramas**. Un proyecto evoluciona en paralelo de distintas formas, y todas ellas quedan controladas.

Definir un **flujo de trabajo en Git** claro, adaptado al proyecto, y que todo el mundo respete es imprescindible. Sin él, el proyecto fracasa.

Un flujo mínimo tiene dos ramas:

Rama	Contenido	Reglas
main	Lo que está **apto para explotación**.	Nadie hace commits directamente en ella. Solo recibe commits ya aprobados de otras ramas.
desarrollo (*dev*)	Lo que está **aprobado para la siguiente entrega** a producción (la siguiente iteración).	Cada commit aporta una funcionalidad o un cambio al sistema.



Para la gestión del proyecto, el repositorio permite **planificar** los cambios, ver **en qué estado** está cada trabajo y **decidir qué se libera** en la siguiente entrega.

> **Ampliación: ramas de trabajo y solicitudes de integración.** En la práctica, cada cambio se desarrolla en una rama propia que sale de *desarrollo* (por ejemplo, `cambio/RED-1234`). Cuando está listo se abre una **solicitud de integración** (*merge request* en GitLab, *pull request* en GitHub). Ese es el punto natural donde se ejecuta el *pipeline* de pruebas, donde otra persona revisa el cambio y donde queda registrada la aprobación. Con ello, el repositorio aporta **evidencias automáticas**: quién propuso el cambio, quién lo aprobó, qué pruebas pasó y qué diferencia exacta introduce. Nombrar las ramas y los commits con la referencia del asunto en Redmine conecta el modelo de trabajo (capítulo 10) con el técnico.

8 Coste, mantenibilidad e inteligencia artificial

8.1 El coste que importa: TCO

Desde el punto de vista de la gestión, lo que cuesta desarrollar o implantar algo pesa mucho menos que lo que cuesta **tenerlo** durante toda su vida: operarlo, mantenerlo, repararlo, sufrir sus caídas. En otras industrias se llama **coste del ciclo de vida** (LCC, *Life Cycle Cost*); en IT, **coste total de propiedad** (TCO, *Total Cost of Ownership*).

Ese es el dato que hay que minimizar. Y para minimizar el TCO puede interesar **aumentar** el coste de desarrollo: invertir en pruebas, automatización y documentación. No se juega a corto plazo.

8.2 Mantenible y evolucionable

Que un programa funcione no tiene nada de especial: se da por descontado. Lo importante es que sea **fácil de mantener y de evolucionar**, porque un producto de software es, por definición, un producto sujeto a cambios y mantenimiento. Lo mismo vale para un *playbook* o para la configuración de una red tratada como código.

8.3 La IA agéntica

Las IA basadas en modelos de lenguaje empezaron como asistentes conversacionales: pregunta y respuesta. Hoy estamos en el **mundo agéntico**: la IA no solo contesta, sino que **escribe programas y los ejecuta**. Y con un programa se puede actuar sobre el mundo físico. Las máquinas que automatizan el trabajo, que antes creaba una persona, ahora las crea una IA.

Aquí hay mucho peligro:

- Las IA se equivocan, cada vez menos, pero siempre se equivocarán.
- El problema de fondo no es la calidad de la IA: es que el trabajo exige **tomar decisiones alineadas** con la persona, la empresa, el proyecto y el cliente, y esa información no se le hace llegar a la IA con un texto.
- Las IA tienden a dar importancia solo a que algo funcione, no a que sea mantenible y evolucionable. **Se comportan como perfiles júnior.**

La pauta de uso es clara: **la persona experta toma las decisiones y la IA es sus manos.** La IA propone; la persona supervisa, decide y corrige, porque es quien va a vivir con esas decisiones. Bien usada, la IA es una ayuda excelente para escribir *playbooks* de Ansible.

9 Implantación en la organización

9.1 Un cambio lento y cultural

Migrar hacia estas formas de trabajo lleva **meses o años**. No hay herramienta mágica: requiere cambios culturales, formación del personal y adaptación de los procesos existentes. Cada paso hay que adaptarlo al contexto, al equipo, al proyecto y a la organización.

Normalmente empieza en **sistemas y operaciones**: es la gente que empieza a usar estas herramientas para su propio trabajo. Hacen su primera instalación, automatizan progresivamente y van introduciendo las herramientas y las formas de trabajo en la empresa. De ahí se expande.

La implantación avanza por fases:

1. Automatizar alguna tarea (los primeros *playbooks*).
2. Someter lo automatizado a control de versiones.
3. Formalizar pruebas automatizadas.
4. Automatizar algunos procesos.

Cuando está montado, se vuela. Pero montarlo es un trabajo enorme.

9.2 Evolución de las estructuras organizativas

Etapa	Modelo	Consecuencias
Años 50-60	**Anarquía**: cada uno a su aire.	Crisis del software.
Desde los 70	**Hipercontrol**: metodologías rígidas (cascada), documentación exhaustiva, departamentos centrales de informática, decisiones centralizadas.	Rigidez, respuesta lenta al cambio, dependencia de procesos y documentos, dificultad para innovar.
Hoy	**Modelos federados o distribuidos**.	Equilibrio entre autonomía de los equipos y coordinación central.

En el modelo federado, los equipos tienen libertad para decidir cómo hacer las cosas, pero se mantiene una coordinación que asegura la coherencia con los objetivos de la empresa, con procedimientos y patrones similares que permiten mover a las personas entre equipos sin perder consistencia. El centro no ordena: **ofrece soporte, pautas, herramientas, plantillas y pipelines de CI/CD**.

Este papel suele tomar forma en un **Centro de Excelencia (CoE)**: un equipo que facilita la adopción de buenas prácticas y la coherencia entre equipos. Normalmente no se crea de golpe: se va fraguando sin que nadie lo decida y al final se formaliza. Solo las organizaciones que parten de cero (los neobancos, por ejemplo) pueden diseñarlo desde el principio.

Estos cambios rompen procedimientos, estructuras organizativas, la forma de tomar decisiones, el reparto del poder y la cultura. Por eso cuestan.

9.3 ITIL

ITIL (*Information Technology Infrastructure Library*) es:

- una colección de **buenas prácticas** para la gestión de servicios IT;
- un **vocabulario común** para hablar de esos servicios;
- una **definición de procesos** (incidencias, problemas, cambios, peticiones…).

No es incompatible con el enfoque ágil. Lo que hay que evitar es aplicarlo como un ejercicio de burocracia. El capítulo 10 recoge la correspondencia entre el modelo de trabajo del NOC y los procesos de ITIL.

10 Caso práctico: migración de ACL

10.1 Planteamiento

Un operador gestionaba las ACL de los routers de unos **3.000 centros de trabajo (más de 6.000 routers)**. Esa gestión se traspasa al equipo del NOC en dos dimensiones:

- **Responsabilidad**: el equipo de trabajo pasa a ser el nuestro.
- **Funcional**: las peticiones de cambio y la revisión de la obsolescencia de las reglas.

El objetivo es migrar las reglas de los routers a los nuevos cortafuegos en **12 meses**.

10.2 Opción 1: enfoque tradicional

Un reparto típico en cascada:

Fase	Peso aproximado
Toma de requisitos	(un mes de documento)
Plan inicial	5 %
Análisis completo del sistema	20 % (2 meses previstos; probablemente 3,5 o más)
Diseño	25 %
Ejecución / implantación	35 %
Pruebas	10 %
Entrega y cierre	5 %

Riesgos que la experiencia ya anticipa:

- **La desviación de una fase arrastra a las demás.** El análisis preocupa especialmente: no se sabe qué se va a encontrar, porque no estaba bajo nuestro control, y hay que mirarlo **router a router**.
- **Los requisitos no están cerrados**: llegarán peticiones de cambio con el tiempo, y cada equipo investigado traerá sorpresas.
- **Retrasos externos**: ventanas de paso a explotación, coordinación con el cliente y otros equipos, burocracia.
- **Las pruebas llegan al final**: hasta entonces no hay garantías de que el trabajo sea correcto.
- **El cliente no obtiene beneficio** hasta pasado al menos el 55 % del tiempo, siendo optimistas; probablemente el 80 %.
- **No se sabe cómo va el proyecto.** El dato objetivo —centros migrados— no existe hasta que empieza la migración, hacia la mitad del proyecto, y ese momento ya se sabe que se retrasará. Para entonces han pasado 6 o 7 de los 12 meses y queda muy poco margen para decidir.

Con este enfoque, la previsión sería empezar a migrar a los 3 meses y repartir los 6.000 routers en los 9 restantes (unos 667 al mes). La letra pequeña: **esas cifras son inventadas**, y en el mes 6 puede haber dos centros migrados.

Lo que ha matado durante décadas a este tipo de proyectos es la **falta de datos reales tempranos** sobre el estado real del sistema, sus «bombas» ocultas y la duración real de las tareas. Los proyectos acababan saliendo, pero tarde, con menos beneficio y con mucho dolor.

10.3 Opción 2: enfoque ágil

El backlog. En lugar de cerrar los requisitos, se abre un *backlog*: el registro de las tareas que *creemos* que habrá que hacer. Es provisional, va a cambiar, y sirve para empezar a organizarse sin pretender organizarlo todo.

> - Migrar las reglas del centro 1 – router 1.
> - Migrar las reglas del centro 1 – router 2.
> - Migrar las reglas del centro 2 – router 1.
> - …
> - Estudiar la capacidad efectiva de los cortafuegos (**prioridad alta**, al siguiente sprint).

La estimación. 6.000 routers en 12 meses son 500 al mes, pero el ritmo no será constante: habrá problemas (la primera vez que aparezca cada uno costará más), los meses no son iguales y el equipo irá ganando experiencia. Lo esperable es que el ritmo crezca con el tiempo. Estimar ahora la duración por router es como leer los posos del café.

El primer sprint. Objetivo: **10 centros, y a ser posible muy distintos entre sí**, para que las sorpresas salgan cuanto antes. Ni siquiera se plantea automatizar todavía: no se conoce el proceso, así que no se puede automatizar. Se trabaja sin más pretensión, sabiendo que el día 1 del mes siguiente quedarán operativos los que se hayan podido terminar. Si son 3, bien; si son 0, también: el objetivo no son los centros.

El objetivo es **una fecha cerrada en la que disponer de información para decidir con menos incertidumbre**:

- se habrán identificado las principales «bombas» y obstáculos (saldrán más; por eso se itera);
- por ejemplo, si se descubre que el operador tarda tres meses en dar información, **se le pregunta ya por todos los centros**;
- el equipo habrá ganado experiencia práctica.

Lo que se busca es un **producto mínimo viable**: un centro migrado de principio a fin.

El compromiso con el cliente. Las fechas comprometidas pueden ser las mismas que en la opción tradicional (1.500 centros a los 6 meses, 3.200 a los 9, 6.000 a los 12), pero con una diferencia radical: **al cabo de un mes hay un informe con datos reales**, y si aparece un problema grave que ponga en riesgo las estimaciones, se comunica en ese momento. Además, es probable que desde el primer mes haya centros migrados en servicio.

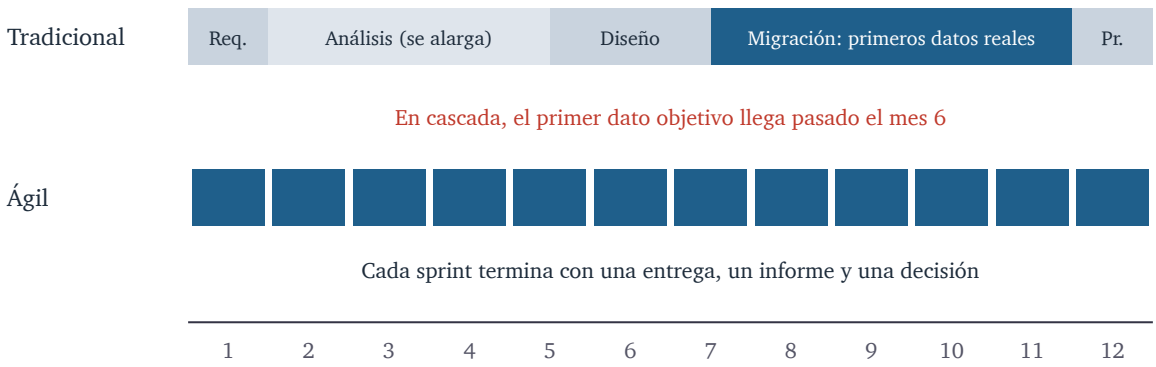


Figura 2. Los dos enfoques pueden comprometer las mismas fechas; lo que cambia es cuándo empieza a haber datos para decidir.

10.4 Las iteraciones siguientes: automatizar

A partir de lo aprendido se empieza a automatizar todo lo posible:

- **Extraer** las reglas actuales de los routers que se van a migrar.
- **Traducir** las reglas del sistema antiguo al nuevo (con Ansible).
- **Configurar** las reglas en los cortafuegos (con Ansible).
- **Probar** que las reglas funcionan (con Ansible).
- **Monitorizar** de forma automatizada el estado de los cortafuegos, con indicadores.
- **Versionar** las reglas de cortafuegos y routers (IaC).

Con las tareas automatizadas se puede abordar el **proceso** completo de migración de un centro:

1. Extraer las reglas del router antiguo.
2. Traducirlas al formato del cortafuegos nuevo.
3. Revisar que las reglas resultantes tienen sentido.
4. Aplicarlas en el cortafuegos, a ser posible primero en un entorno de pruebas.
5. Verificar que funcionan correctamente en ese entorno.
6. Aplicarlas en explotación.
7. Probar que todo funciona en producción.
8. Tener planificada una **vuelta atrás de ejecución automatizada**, apoyada en el control de versiones.
9. Retirar las reglas de los routers (al menos algunas).
10. Probar que los routers han quedado bien.
11. Documentar el proceso y los cambios realizados.

Habrá reglas que se puedan extraer o migrar automáticamente y otras que no; esas se harán a mano. Aun así, el conjunto avanza de forma más eficiente, consistente y con menos errores.

10.5 El papel de la gestión

Con el gorro de gestión del proyecto, **no se hacen esas automatizaciones: se impulsa su creación.** Es una aplicación del principio de **separación de responsabilidades** (*Separation of Concerns*, SoC): cada cual se ocupa de su plano, y la gestión se ocupa de que el trabajo técnico exista, avance y sea visible.

> **Ampliación: definición de «hecho» para un centro.** Para que los números del cuadro de mando signifiquen algo, conviene acordar desde el primer sprint qué significa que un centro esté **migrado**. Una definición razonable: reglas extraídas y versionadas en Git; reglas traducidas y revisadas; aplicadas en el cortafuegos; pruebas positivas y negativas superadas en explotación; reglas retiradas del router y comprobado su estado; asunto de Redmine cerrado con evidencias enlazadas (commit, ejecución del *playbook*, informe de pruebas). Un centro que no cumple todo esto está **en proceso**, no migrado.

11 Gestión de riesgos

11.1 Cálculo del riesgo

Hay que identificar cuanto antes los problemas potenciales y **tasarlos**:

> $$Riesgo = Probabilidad\ de\ que\ ocurra\ el\ problema \times Impacto\ (consecuencias)\ del\ problema$$

El riesgo ayuda a **priorizar tareas** y a **tomar decisiones**.

Ampliación: matriz de riesgos. Una forma práctica de aplicar la fórmula es puntuar probabilidad e impacto de 1 a 5 y multiplicar. Con ello se obtiene una prioridad comparable entre riesgos de naturaleza muy distinta:

	Impacto 1	Impacto 2	Impacto 3	Impacto 4	Impacto 5
Probabilidad 5	5	10	15	**20**	**25**
Probabilidad 4	4	8	12	**16**	**20**
Probabilidad 3	3	6	9	12	**15**
Probabilidad 2	2	4	6	8	10
Probabilidad 1	1	2	3	4	5

Cuando el impacto es catastrófico, el riesgo se trata aunque la probabilidad parezca baja, como en el ejemplo siguiente.

11.2 Ejemplo: la capacidad de los cortafuegos

Si no se sabe si los cortafuegos van a aguantar el tráfico de todos los centros migrados, puede caerse todo más adelante. **Da igual la probabilidad: el impacto es enorme.** Hay que resolverlo pronto, y eso se traduce en tareas de alta prioridad:

- Migrar muchos centros que pasen por el mismo cortafuegos, eligiendo un caso en el que la migración y las pruebas sean sencillas.
- Simular tráfico a través de los cortafuegos: pruebas de **rendimiento**, de **carga** y de **estrés**.

Si el cortafuegos ya está en servicio, la prueba aumenta el riesgo y obliga a decidir: no usar el equipo real, hacer la prueba fuera de horario, comprar otro equipo, o programar una **ventana de indisponibili-**

dad comunicada con antelación («el día 17, de 9 a 10, no habrá red», avisando a todo el mundo dos semanas antes).

Ampliación: registro de riesgos del proyecto. Un primer registro, a revisar en cada sprint:

Riesgo	P	I	Respuesta
Los cortafuegos no soportan la carga de todos los centros.	2	5	Pruebas de rendimiento, carga y estrés tempranas; plan de capacidad.
Reglas sin documentar o sin propietario conocido en los routers.	5	3	Extracción automatizada y revisión de obsolescencia; consulta temprana al propietario.
El operador saliente tarda en dar información.	4	4	Pedir la información de todos los centros en el primer sprint; registrar el tiempo de respuesta como métrica.
La traducción abre más de lo que había.	3	5	Pruebas negativas obligatorias antes de cerrar un centro.
Falta de ventanas de cambio.	3	3	Calendario de ventanas acordado por adelantado; agrupar centros por ventana.
Conocimiento concentrado en pocas personas.	3	4	*Playbooks* versionados y documentados; revisión cruzada de cambios.

12 El modelo de trabajo del NOC

Las tareas de bajo nivel se controlan con una herramienta: **Redmine**, **GitLab** u otra. Para que esa herramienta sirva para gestionar —y para que el cuadro de mando que se construya sobre ella diga la verdad—, el trabajo tiene que estar bien modelado. El documento *Modelo de trabajos vivos en un NOC* lo desarrolla en detalle; aquí se resume.

12.1 Los cinco planos

> *Un asunto no se gestiona solo por su estado. Se gestiona por la combinación de tipo, estado, atributos, evidencias y relaciones.*

Plano	Pregunta que responde
Tipo	¿Qué es este asunto?
Estado	¿Dónde está ahora mismo dentro del flujo?
Atributos	¿Qué datos lo caracterizan? (prioridad, impacto, urgencia, riesgo, centro, responsable, SLA…)
Evidencias	¿Qué demuestra que ha avanzado o se ha cerrado? (aprobación, log, resultado de prueba, commit, diff…)
Relaciones	¿Con qué otros elementos está conectado? (ticket, cambio, commit, *playbook*, prueba…)

12.2 Tipos de trabajo

Tipo	Qué es	Pregunta clave
Proyecto / iniciativa	Contenedor con objetivo, alcance, varias piezas y un fin identificable.	¿Tiene un objetivo amplio, varias piezas y un final?
Petición de servicio	Alguien solicita algo que el servicio debe gestionar; no hay nada roto.	¿Alguien pide algo que hay que tramitar?
Cambio	Modificación controlada sobre producción. Subtipos: estándar, normal, mayor, de emergencia.	¿Modifica producción o puede afectar al servicio?
Incidencia	Algo no funciona como debería; hay que restaurar el servicio.	¿Hay interrupción o degradación?
Problema	Investigación de la causa raíz para evitar que se repita.	¿Hay que averiguar por qué pasa?
Mejora	Aumenta calidad, eficiencia, seguridad o visibilidad sin urgencia inmediata.	¿Mejora el servicio aunque no sea urgente?
Tarea	Unidad concreta de trabajo, dentro de alguno de los anteriores.	¿Qué acción concreta hay que hacer ahora?

En el proyecto de migración, la migración en su conjunto es un **proyecto**; cada centro, un **cambio** (normalmente normal, y estándar cuando el procedimiento esté probado y automatizado); y cada paso del proceso, una **tarea**. Una caída provocada por una migración es una **incidencia**, y si se repite en varios centros abre un **problema**.

12.3 Estados y bloqueo

Estados habituales: nuevo, en análisis / clasificación, pendiente de información, priorización pendiente, pendiente de aprobación, planificado, en ejecución, en validación, resuelto, cerrado, cancelado.

Bloqueado no es un tipo: es un estado transversal que puede aparecer en casi cualquier punto del flujo. Un asunto bloqueado debería tener siempre motivo y responsable de desbloqueo.

> *Un bloqueo visible se ataca. Un bloqueo invisible envejece.*

Muchas transiciones pueden **automatizarse** a partir de eventos de las herramientas: se lanza el *playbook* → en ejecución; termina → en validación; pasan las pruebas → resuelto. Así los estados dejan de ser burocracia y se convierten en visibilidad.

12.4 Equivalencias con Redmine

Concepto del modelo	Representación posible en Redmine
Tipo de asunto	Tracker, categoría, proyecto, tarea padre o campo personalizado
Proyecto / iniciativa	Proyecto, tarea padre, versión o hito
Petición, cambio, incidencia, problema, mejora	Petición (*issue*) con su tracker correspondiente
Tarea	Subtarea o petición relacionada
Estado	Flujo de trabajo (*workflow*) del tracker
Prioridad	Campo prioridad
Impacto, urgencia, riesgo	Campos personalizados
Bloqueo	Estado, etiqueta o campo específico
Motivo y responsable de desbloqueo	Campos personalizados o asignación
Evidencias	Adjuntos, comentarios, enlaces
Relaciones	Relaciones entre peticiones, enlaces externos, referencias a commits

12.5 Equivalencias con ITIL

Modelo operativo	Correspondencia en ITIL
Petición de servicio	Gestión de peticiones de servicio
Incidencia	Gestión de incidencias
Problema	Gestión de problemas
Cambio	Habilitación / gestión de cambios
Mejora	Mejora continua
Proyecto / iniciativa	Gestión de proyectos / planificación de la evolución del servicio
Tarea	Actividad operativa dentro de un proceso
Bloqueo	Impedimento, dependencia o espera dentro del flujo
Evidencia	Registro, prueba, validación, aprobación o documentación de cierre

Este mapeo evita mezclar conceptos: una petición no es automáticamente una incidencia, una incidencia no es un problema, un cambio no es simplemente una tarea y una mejora no debería desaparecer por no ser urgente.

13 Cuadros de mando

13.1 Datos no es entender

En herramientas como Redmine o GitLab se acumula una cantidad enorme de datos. Pero **tener datos no es entender los datos**. Lo que ayuda a entender conjuntos de datos es la estadística, y la herramienta de gestión que lo concreta es el **cuadro de mando**.

Un cuadro de mando **no es una colección de gráficas e indicadores**, como quien colecciona cromos. Es una herramienta para:

- dar **visibilidad** sobre el trabajo que hay;
- informar del **ritmo** de trabajo;
- identificar **problemas y desviaciones** respecto a lo esperado o planificado.

Y todo cuadro empieza por una pregunta: **¿a qué preguntas queremos responder?** Si una gráfica no responde a ninguna pregunta útil, sobra.

13.2 Cuadro operativo y BI

	Objetivo	Metáfora
Cuadro de mando operativo	Ver el estado del servicio o del proyecto y decidir qué hacer ahora.	La cabina: sirve para conducir.
BI / análisis	Cruzar variables, buscar causas y patrones, mejorar a medio y largo plazo.	El taller: sirve para entender y mejorar.

13.3 Tres niveles de información

Nivel	Pregunta	Ejemplos
Cantidades (estado)	¿Qué hay ahora?	Abiertos, por tipo, por estado, bloqueados.
Cantidades (ritmo) y tiempos	¿Cómo se mueve?	Entradas y salidas por semana; tiempo en cada estado; antigüedad.
Análisis	¿Por qué pasa?	Bloqueos por causa; retrasos por proveedor; fallos por tipo de cambio.

> *Las cantidades me dicen que tengo trabajo pendiente. Los tiempos me dicen si ese trabajo está vivo o se está pudriendo.*

13.4 Métricas automáticas y de disciplina

- **Métricas automáticas**: las genera la propia maquinaria técnica (Git, *pipelines*, Ansible / AAP, Terraform, pruebas, monitorización). La herramienta no opina: registra lo que ha pasado.
- **Métricas de disciplina**: dependen de que las personas usen bien el modelo (estados al día, bloqueos marcados, evidencias adjuntas). Son necesarias, pero mienten si el modelo no está adoptado.

Al principio conviene apoyarse en las métricas automáticas e ir incorporando las de disciplina a medida que el modelo se consolida.

13.5 El cuadro de mando del proyecto de migración

13.5.1 Datos clave: la foto

Para cada uno de los tres niveles de agregación —**grupos**, **centros** y **equipos (routers)**—, el número de elementos en cada situación:

	Grupos	Centros	Equipos
Totales			
Migrados			
En proceso			
Pendientes			
Enquistados			

Acompañado de una **tabla con los cinco elementos más preocupantes**: los que más tiempo llevan enquistados.

Esta foto da visibilidad del estado actual del proyecto, del trabajo hecho y del que está en curso. Pero no dice nada del ritmo.

13.5.2 Ritmo

- **Series temporales** de cada uno de los datos anteriores.
- **Indicadores de variación** respecto al periodo anterior (por ejemplo, «+3 respecto al mes pasado»).
- **Proyección**: con el ritmo actual, ¿se llega o no se llega a la fecha comprometida?
- **Tiempo por unidad de trabajo**: cuánto se invierte actualmente por grupo, ACL, centro y router, y **cómo evoluciona**. La foto no basta: interesa la tendencia, que debería mostrar el aprendizaje del equipo.

Ampliación: proyección sencilla. Con el ritmo medio de las últimas n semanas (r, en centros por semana) y los centros pendientes (P), las semanas necesarias son P / r. Si se compara con las semanas que quedan hasta la fecha comprometida, se obtiene una señal clara: en plazo, en riesgo o fuera de plazo. El ritmo necesario para llegar es P / *semanas restantes*. Mostrar a la vez el ritmo **real** y el **necesario** es una de las gráficas más útiles del cuadro: cuando las dos líneas se separan, hay que decidir algo.

13.5.3 Que el dato sea una prueba

¿De dónde sale el «número de reglas migradas»? Si sale de Redmine porque un compañero lo ha anotado, **es una ruina**: es una métrica de disciplina, subjetiva y sin evidencia. Ese dato debe ser el **resultado de una prueba**, a ser posible automatizada. Es la aplicación directa de «el sistema funcionando es la medida principal del progreso».

13.5.4 Control de riesgos

- **Estado de los cortafuegos** (carga de trabajo), con su evolución.
- **Tiempo de respuesta de terceros** de los que se depende.
- **SLI, SLO, SLA y presupuesto de indisponibilidad.**

Ampliación: SLI, SLO, SLA y presupuesto de error.

Término	Qué es	Ejemplo
SLI (*Service Level Indicator*)	La medida: un indicador objetivo del nivel de servicio.	Porcentaje de comprobaciones de conectividad de un centro que tienen éxito.
SLO (*Service Level Objective*)	El objetivo interno para ese indicador.	99,9 % mensual.
SLA (*Service Level Agreement*)	El compromiso contractual con el cliente, con consecuencias si se incumple. Suele ser menos exigente que el SLO para tener margen.	99,5 % mensual.
Presupuesto de error	La indisponibilidad que el SLO permite: 100 % − SLO.	0,1 % de un mes de 30 días ≈ 43 minutos.

El presupuesto de error convierte la fiabilidad en un recurso que se gasta: cada ventana de cambio y cada incidencia lo consumen. Si una migración se come el presupuesto del mes, hay que frenar el ritmo de cambios y reforzar las pruebas antes de seguir. Es el puente entre el ritmo del proyecto y la estabilidad del servicio.

13.5.5 Dos esfuerzos permanentes

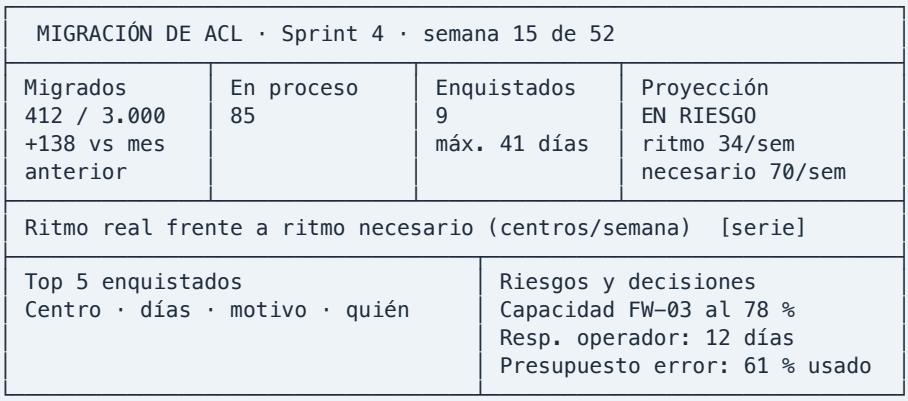
Allí donde sea posible, hay que poner un esfuerzo enorme en dos cosas:

1. **Automatizar la generación del cuadro de mando**, para que esté en **tiempo real**.
2. **Controlar la calidad de los datos**, para reducir la incertidumbre.

13.6 Un cuadro para cada audiencia

Audiencia	Pregunta principal
Equipo	¿Qué tenemos que hacer hoy?
Gestión del servicio	¿Cómo va el servicio y dónde debo intervenir?
Cliente o comité	¿Qué se ha hecho, qué queda, qué riesgos hay y qué decisiones necesitamos?
Análisis / BI	¿Qué está pasando en el sistema y cómo podemos mejorarlo?

Ampliación: boceto del cuadro para el cliente.



Las cifras son ilustrativas. La estructura responde, de arriba abajo, a las tres preguntas: qué hay, cómo se mueve y qué hay que decidir.

13.7 Reglas de diseño

1. **Debe responder preguntas concretas.** Empieza por la pregunta, no por la gráfica.
2. **Debe permitir actuar.** Si no ayuda a decidir, es decoración.
3. **Debe distinguir estado, ritmo y análisis.**
4. **Debe tener datos fiables.** Si los tipos están mal, los estados sin actualizar o los cierres sin evidencia, el cuadro miente.
5. **Debe evitar las métricas de vanidad**: totales históricos, número de comentarios, porcentajes de avance subjetivos.
6. **Las métricas no deben usarse contra las personas.** Cuando una métrica se usa para castigar, deja de medir la realidad y empieza a medir la capacidad de la gente para protegerse de ella.

13.8 Aterrizaje en Redmine, por fases

Fase	Contenido	Dificultad en Redmine
1. Cantidades	Asuntos por tipo, estado, abiertos, cerrados, bloqueados; creados frente a cerrados.	Directa: consultas y filtros.
2. Antigüedad	Asuntos más antiguos, bloqueos antiguos, sin actualización reciente.	Sencilla, por fechas de creación y actualización.
3. Tiempos por estado	Tiempo medio en análisis, en aprobación, bloqueado, en validación; tiempo de ciclo.	Requiere explotar el histórico, *plugins* o exportar a BI.
4. Integración	Git, *pipelines*, Ansible / AAP, pruebas y monitorización alimentando el cuadro.	Requiere integración y automatización.

> *El cuadro de mando también evoluciona. No se implanta perfecto el primer día.*

14 Ideas clave

1. **La pregunta de la gestión es «¿cómo voy?»**, y solo se contesta bien con datos objetivos y tempranos.
2. **Los requisitos cambian.** La cascada lo sufre; el enfoque ágil lo asume con entregas incrementales y fechas que no se mueven.
3. **El sistema funcionando es la medida del progreso, y quien dice que funciona son las pruebas.**
4. **Entregar a menudo multiplica las pruebas y los despliegues.** Solo la automatización lo hace sostenible.
5. **En TIC, automatizar es programar.** Primero tareas; después, procesos. Automatizar es caro y exige conocimiento profundo.
6. **IaC es tratar la infraestructura como código**: versionada, declarativa e idempotente, lo que permite moverse entre versiones con seguridad.
7. **Git y un flujo de ramas claro** son la base del control de cambios y una fuente de evidencias automáticas.
8. **El coste que importa es el TCO.** Invertir en calidad durante el desarrollo abarata la vida del sistema.
9. **La IA son las manos; las decisiones son de la persona.**
10. **La implantación es lenta y cultural**, empieza en operaciones y tiende a modelos federados con un centro de excelencia.
11. **En la migración de ACL, el primer sprint compra información**, no centros migrados.
12. **Riesgo = probabilidad × impacto.** Lo catastrófico se ataca pronto aunque sea improbable.
13. **El cuadro de mando nace del modelo de trabajo**: tipos, estados, atributos, evidencias y relaciones bien definidos.
14. **Un cuadro de mando no consiste en poner gráficas. Consiste en construir una conversación de gestión basada en datos.**

15 Glosario

Término	Definición
ACL	Lista de control de acceso: conjunto de reglas que permiten o deniegan tráfico.
Backlog	Registro priorizado y provisional del trabajo que se cree que hay que hacer.
CI / CD	Integración continua / entrega o despliegue continuo.
CoE	Centro de Excelencia: equipo que da soporte, pautas, herramientas y plantillas al resto.
Commit	Fotografía completa de un proyecto en un momento dado, en Git.
Declarativo	Paradigma en el que se describe el estado deseado, no los pasos para alcanzarlo.
Defecto	Huella que deja un error humano en el producto.
Fallo	Desviación del comportamiento esperado al usar el producto.
FIRST	Principios de buenas pruebas: rápidas, independientes, repetibles, autovalidables y oportunas.
IaC	Infraestructura como código.
Idempotencia	Propiedad de un programa que, desde cualquier estado inicial, lleva siempre al mismo estado final.
Iteración / sprint	Periodo de duración fija que termina en una entrega al cliente.
ITIL	Marco de buenas prácticas, vocabulario y procesos para la gestión de servicios IT.
MVP / PMV	Producto mínimo viable: lo mínimo que permite aprender con datos reales.
NetDevOps	Cultura DevOps aplicada a la operación de redes.
Pipeline	Programa que encadena automáticamente las tareas de un proceso (preparar entorno, aplicar, probar, informar).
Rollback	Vuelta a una versión anterior conocida y estable.
Semver	Versionado semántico: MAJOR.MINOR.PATCH.
SLI / SLO / SLA	Indicador, objetivo y acuerdo de nivel de servicio.
Smoke test	Prueba de humo: comprobación rápida de lo esencial tras un despliegue.
SoC	Separación de responsabilidades.
TCO	Coste total de propiedad de un sistema durante su vida.